In [1]:
```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.metrics import mean_absolute_error
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

In [2]:
```
FEATURES = [
    "distance_km",
    "prep_time_min",
    "traffic_level",
    "rain",
]

TARGET = "delivery_min"
```

In [4]:
```
import os

print(os.getcwd())
print(os.listdir("../data"))
```

```
C:\MLOps\P03-choosing-model-honestly
['delivery_times.csv']
```

In [7]:
```
import os

print("Current folder:")
print(os.getcwd())

print("\nFiles/folders one level up:")
print(os.listdir(".."))

print("\nFiles in data folder:")
print(os.listdir(r"C:\MLOps\data"))
```

```
Current folder:
C:\MLOps\P03-choosing-model-honestly

Files/folders one level up:
['.git', '.ipynb_checkpoints', '.venv', 'data', 'P01-workbench', 'P02-first-mode
l', 'P03-choosing-model-honestly', 'README.md', 'requirements-lock.txt', 'require
ments-tools.txt', 'SETUP.md']

Files in data folder:
['delivery_times.csv']
```

In [8]:
```
import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"
```

```
def make_delivery_csv(path=DATA):
    rng = np.random.default_rng(SEED)

    distance_km = np.round(
        rng.uniform(0.5, 12.0, N_ROWS), 2
    )

    prep_time_min = np.round(
        rng.uniform(5, 30, N_ROWS), 0
    )

    traffic_level = rng.integers(1, 4, N_ROWS)

    rain = rng.binomial(1, 0.25, N_ROWS)

    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )

    path.parent.mkdir(parents=True, exist_ok=True)

    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)

        w.writerow([
            "distance_km",
            "prep_time_min",
            "traffic_level",
            "rain",
            "delivery_min"
        ])

        for i in range(N_ROWS):
            w.writerow([
                distance_km[i],
                int(prep_time_min[i]),
                int(traffic_level[i]),
                int(rain[i]),
                delivery_min[i]
            ])

    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ..\data\delivery_times.csv
```

In [9]:
```
df = pd.read_csv(DATA)
```

```
X = df[FEATURES]
y = df[TARGET]

print(df.head())
print("Dataset shape:", df.shape)
```

```
   distance_km  prep_time_min  traffic_level  rain  delivery_min
0         9.40             17              1     0          51.3
1         5.55             24              2     1          54.2
2        10.37             28              3     0          67.7
3         8.52             23              2     0          51.2
4         1.58             29              2     1          42.1
Dataset shape: (600, 5)
```

In [10]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Development data:", X_train.shape)
print("Final test data:", X_test.shape)
```

```
Development data: (480, 4)
Final test data: (120, 4)
```

In [11]:
```
model = DecisionTreeRegressor(
    max_depth=12,
    random_state=42
)

model.fit(X_train, y_train)
```

Out[11]:

▾ DecisionTreeRegressor

▸ Parameters

▸ Fitted attributes

In [12]:
```
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)
```

In [13]:
```
T1_train_mae = mean_absolute_error(y_train, train_predictions)
T1_test_mae = mean_absolute_error(y_test, test_predictions)
T1_gap = T1_test_mae - T1_train_mae

print("T1 Train MAE:", T1_train_mae)
print("T1 Test MAE:", T1_test_mae)
print("T1 Gap:", T1_gap)
```

```
T1 Train MAE: 0.04695601851851851
T1 Test MAE: 3.397986111111111
T1 Gap: 3.3510300925925924
```

In [14]:
```
cv = KFold(
    n_splits=5,
    shuffle=True,
```

```
    random_state=42
)
```

In [15]:
```
depths = [2, 4, 6, None]

T2_scores = {}

for depth in depths:
    model = RandomForestRegressor(
        n_estimators=50,
        max_depth=depth,
        random_state=42
    )

    scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring="neg_mean_absolute_error"
    )

    mae_scores = -scores
    T2_scores[depth] = mae_scores.mean()

print(T2_scores)
```

```
{2: np.float64(5.405008168973727), 4: np.float64(3.588674460101925), 6: np.float6
4(2.8570225653528234), None: np.float64(2.7736583333333322)}
```

In [16]:
```
T2_best_depth = min(T2_scores, key=T2_scores.get)

print("T2 Best Depth:", T2_best_depth)
print("Best CV MAE:", T2_scores[T2_best_depth])
```

```
T2 Best Depth: None
Best CV MAE: 2.7736583333333322
```

In [17]:
```
def compare(models):
    results = {}

    for model_name, estimator in models.items():
        scores = cross_val_score(
            estimator,
            X_train,
            y_train,
            cv=cv,
            scoring="neg_mean_absolute_error"
        )

        mae_scores = -scores

        results[model_name] = {
            "cv_mae_mean": mae_scores.mean(),
            "cv_mae_std": mae_scores.std()
        }

    return results
```

In [18]:
```
models = {
    "LinearRegression": LinearRegression(),

    "DecisionTree_depth4": DecisionTreeRegressor(
        max_depth=4,
        random_state=42
    ),

    "RandomForest_50": RandomForestRegressor(
        n_estimators=50,
        random_state=42
    )
}
```

In [19]:
```
comparison_results = compare(models)

comparison_results
```

Out[19]:
```
{'LinearRegression': {'cv_mae_mean': np.float64(2.064247868929564),
  'cv_mae_std': np.float64(0.23299464934509947)},
 'DecisionTree_depth4': {'cv_mae_mean': np.float64(4.407235989394573),
  'cv_mae_std': np.float64(0.24497107737403037)},
 'RandomForest_50': {'cv_mae_mean': np.float64(2.7736583333333322),
  'cv_mae_std': np.float64(0.2310832351394196)}}
```

In [20]:
```
final_model = LinearRegression()
```

In [21]:
```
final_model.fit(X_train, y_train)
```

Out[21]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

In [22]:
```
final_predictions = final_model.predict(X_test)

final_mae = mean_absolute_error(
    y_test,
    final_predictions
)

print("Final Test MAE:", final_mae)
```

```
Final Test MAE: 1.924653079833835
```